

Utilizatori în Online School

Opt teme de moștenire și polimorfism, pe proiectul tău · Java · Adrian

Unde lucrezi: în repo-ul tău `online-school-vanilla-java`, peste codul care există deja. Aplicația trebuie să ruleze la fel de bine după fiecare temă — nu la final.

Ce ai voie: tot ce folosești deja (fișiere, `Optional`, `try`, `List`), plus `extends`, `super(...)`, `super.metoda()`, `@Override`, câmpuri `protected`.

Încă NU folosim: `abstract`, `interface`, clase generice scrise de tine, stream-uri, biblioteci noi.

Cum se verifică: fiecare temă are un rând „Gata când”. Toate se verifică **rulând** aplicația și uitându-te la ce iese, plus la ce scrie în fișierele `.txt` — nu prin „compilează, deci merge”.

Partea 1 — ierarhia

1. Scoate ce e comun într-o clasă `User`

nivel 1

Astăzi `Student` ține și date care n-au nimic de-a face cu faptul că e student: identificator, nume, prenume, email, parolă. Creează o clasă `User` care să țină partea asta, și lasă în `Student` doar ce este cu adevărat al unui student.

Restul aplicației — `StudentRepository`, `View`, `Course`, `Enrollment` — rămâne cum e și continuă să funcționeze. Tu decizi ce câmpuri urcă, ce constructori are fiecare nivel și cum se împarte între ele linia citită din fișier.

Gata când: pornești aplicația, listezi cărțile și înscrierile ca înainte, iar `students.txt` arată exact ca înainte de refactorizare.

2. Adaugă al doilea tip de utilizator: `Teacher`

nivel 1

Un profesor are aceleași date de identificare ca oricine intră în aplicație, plus ce e specific meseriei lui. Alege tu ce anume (cel puțin două câmpuri proprii) și construiește-l din aceeași formă de linie ca ceilalți utilizatori.

Gata când: creezi în cod un `Student` și un `Teacher`, le pui pe amândouă într-o listă de `User` și le afișezi din aceeași buclă.

3. Fișierul trebuie să spună ce tip de utilizator e pe fiecare linie

nivel 1

Salvarea și citirea merg acum pe un singur tip. Din clipa în care ai două, o linie dintr-un fișier nu mai e suficientă ca să știi ce obiect trebuie construit înapoi în memorie.

Fă în așa fel încât ce se scrie pe disc să fie de-ajuns pentru a reconstrui exact tipul potrivit, și verifică drumul întreg: obiect → fișier → obiect.

Gata când: salvezi un profesor, **oprești aplicația**, o pornești din nou — și profesorul e tot acolo, ca `Teacher`, nu ca altceva și nu dispărut.

4. TeacherRepository

nivel 1

Profesorii au nevoie de aceleași operații pe care le au studenții: toate, după id, după email, adăugare cu email unic, modificare, ștergere, salvare pe disc. Scrie depozitul lor și fișierul de date aferent, respectând contractul pe care l-ai folosit deja la studenți.

Gata când: adaugi doi profesori, îl ștergi pe unul, repornești aplicația și în fișier a rămas exact cine trebuia. Încerci să adaugi un profesor cu un email deja existent și aplicația refuză, fără să strice fișierul.

Partea 2 — polimorfismul care face muncă

5. Autentificare pentru oricine

nivel 2

Scrie autentificarea (email + parolă) o singură dată, peste o listă de `User`, astfel încât să funcționeze pentru ambele tipuri de utilizatori. Metoda care verifică datele nu are voie să întrebe ce tip de utilizator primește.

Gata când: te loghezi o dată ca student și o dată ca profesor, prin **aceeași** metodă, și în ambele cazuri aplicația știe cine ești. Cu o parolă greșită te refuză, fără excepție aruncată în consolă.

6. Meniul arată altceva pentru student și altceva pentru profesor

nivel 2

După autentificare, studentul vede opțiunile lui (înscriserile, cursurile disponibile, cărțile), iar profesorul le vede pe ale lui (cursurile pe care le predă, studenții înscriși la ele).

Constrângere: în `View` nu ai voie să scrii `instanceof` și nici vreun câmp de tip „rol” pe care să-l compari cu `if`. Diferența o decide obiectul autentificat, nu meniul.

Gata când: rulezi de două ori, cu doi utilizatori diferiți, și vezi două meniuri diferite. Cauți `instanceof` în `View` și nu găsești niciunul.

7. Fiecare curs are un titular

nivel 2

Un curs este predat de un profesor. Leagă-le, și arată titularul acolo unde afișezi cursurile disponibile.

Gândește-te ce se întâmplă cu un curs al cărui titular încă nu a fost stabilit — aplicația nu are voie să cadă din cauza asta.

Gata când: „arată cursurile disponibile” tipărește, pentru fiecare curs, numele titularului, iar un curs fără titular se afișează și el, cu un mesaj clar în locul numelui.

Partea 3 — provocare

8. Un singur raport pentru toată școala

provocare

Adaugă în meniul o opțiune care tipărește **toți** utilizatorii aplicației, în aceeași listă: fiecare cu datele care-l descriu — studentul cu ale lui, profesorul cu ale lui.

Constrângere: metoda care tipărește raportul are **o singură buclă**, niciun `if` după tipul utilizatorului și niciun `instanceof`. Dacă mâine adaugi un al treilea tip de utilizator, raportul nu are voie să se modifice deloc.

Gata când: raportul iese corect pentru studenți și profesori; apoi, ca probă, adaugi în cod încă un tip de utilizator, îl pui în listă, și raportul îl afișează corect **fără** ca tu să fi atins metoda de raport.